ORF409 HW5 (Ch8)

(Q.1) $\sum_{i=1}^{n}(x_i-\bar{x})^2 = \sum_{i=1}^{n}(x_i^2-2x_i\bar{x}+\bar{x}^2)$

$$= \sum_{i=1}^{n} x_i^2 - \sum_{i=1}^{n} 2x_i\bar{x} + \sum_{i=1}^{n}\bar{x}^2$$

$$= \sum_{i=1}^{n} x_i^2 - 2\bar{x}\sum_{i=1}^{n} x_i + n\bar{x}^2$$

$$= \sum_{i=1}^{n} x_i^2 - 2\bar{x}(n\bar{x}) + n\bar{x}^2$$

$$= \sum_{i=1}^{n} x_i^2 - 2n\bar{x}^2 + n\bar{x}^2$$

$$= \sum_{i=1}^{n} x_i^2 - n\bar{x}^2$$

— QED

(Q.2) Let X = R.V taking values $x_1, \ldots, x_n$

Random Variable

So, $\cancel{\sum_{i=1}^{n}}$ $\text{var}(X) = E(X^2) - (E(X))^2$ → Here $\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$

$$\Rightarrow \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2 = \frac{1}{n}\sum_{i=1}^{n} x_i^2 - \left(\frac{1}{n}\sum_{i=1}^{n} x_i\right)^2$$

$$\Rightarrow \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2 = \frac{1}{n}\sum_{i=1}^{n} x_i^2 - \bar{x}^2$$

$$\Rightarrow \sum_{i=1}^{n}(x_i - \bar{x})^2 = \sum_{i=1}^{n} x_i^2 - n\bar{x}^2$$

- QED

ORF 409 HW5 ~~Rough~~

(Q.4)
(a) Note that if $X_1, X_2, \ldots, X_n$ are iid std normal samples

$N(\mu, \sigma^2)$, $\mu \to 0$, $\sigma^2 \to 1$

& $\bar{X} = \frac{\sum X_i}{n}$,

$$\frac{\bar{X} - E(\bar{X})}{\sqrt{Var(\bar{X})}} \Rightarrow N(0,1)$$

$$\Rightarrow \frac{\bar{X} - E(\bar{X})}{\sqrt{\frac{1}{n^2} \cdot n \, Var(X_i)}} \Rightarrow N(0,1)$$

$Var(X_i) \to \sim s^2$

So, $\frac{s}{\sqrt{n}} < 0.1$

$\Rightarrow \frac{\sigma}{\sqrt{n}} < 0.1$ ($\because$ $s^2 (= \frac{1}{n-1} \sum (x_i - \bar{x})^2)$ is an unbiased estimator of $var(X_i) = \sigma^2$, $s \xrightarrow{w.ar^m} \sigma$)

$\Rightarrow \frac{1}{\sqrt{n}} < 0.1$

$\Rightarrow \sqrt{n} > 10$

$\Rightarrow n > 100$

~~$\Rightarrow$ (We expect 100 numbers)~~

$\Rightarrow$ we expect ~~101 numbers~~ $\sim > 100$ numbers

~ 100 numbers (101, 102 maybe)

Q. 9)

(a) $P(M>n) = P(U_1 \le U_2 \le \text{—} \le U_n)$

~~n! ways to arrange $[U_1, U_2, \text{—}, U_n]$ each equally likely~~

$[U_1, U_2, \text{—}, U_n]$ are n iid $U[0,1]$ draws.

Since they are (independent) & (identically distributed), each of the n! possibilities (of arranging these from smallest to largest) is equally likely.

$n! \Rightarrow [U_1, U_2, \text{—}, U_{n-1}, U_n] \rightarrow$ This means: $U_1 \le U_2 \le \text{—} \le U_{n-1} \le U_n$

$U_1, U_2, \text{—}, U_n, U_{n-1}]$

$|$

$[U_n, U_{n-1}, \text{—}, U_2, U_1]$

$\Rightarrow P([U_1, U_2, \text{—}, U_{n-1}, U_n]) = \frac{1}{n!}$

$\Rightarrow P(U_1 \le U_2 \le \text{—} \le U_{n-1} \le U_n) = \frac{1}{n!}$

$\Rightarrow P(M>n) = \frac{1}{n!}$

- QED.

(b) $E(M) = \sum_{n=0}^{\infty} P(M>n)$

$= \sum_{n=0}^{\infty} \frac{1}{n!}$

$= \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \dots$

$= e^1$ ~~(by Taylor expansion~~

$= e$ (By Taylor series expansion:

$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$

$\Rightarrow e^1 = \sum_{n=0}^{\infty} \frac{1^n}{n!} = \sum_{n=0}^{\infty} \frac{1}{n!}$)

(Q.16)

M2: Samples $\rightarrow$ d_1 d_2 ——— d_k

n_1 n_2 n_k

M2:

$$\text{mean} = \frac{1}{k}\left(\frac{d_1}{n_1} + \frac{d_2}{n_2} + \cdots + \frac{d_k}{n_k}\right)$$

$$= \frac{d_1}{n_1}\left(\frac{1}{k}\right) + \cdots + \frac{d_k}{n_k}\left(\frac{1}{k}\right)$$

wts: equal to each cycle but NOT for each person

same for

C1 person $\Rightarrow$ wt $= \frac{1}{n_1 k}$

C_2 — || — $\frac{1}{n_2 k}$

M3:

$$\text{mean} = \frac{d_1 + d_2 + \cdots + d_k}{n_1 + n_2 + \cdots + n_k}$$

$$= \frac{d_1}{n_1}\left(\frac{n_1}{\Sigma n_i}\right) + \cdots + \frac{d_k}{n_k} \cdot \frac{n_k}{(\Sigma n_i)}$$

same wt for each person but not for each cycle

$\Rightarrow$ cycle$_i$ wt $\propto n_i$

since we want avg time "a person" spends in the system, giving equal wt to each person makes more sense.

$\Rightarrow$ M3 ✓

Now, MSE $= E_F(g(\bar{x}) - \theta(F))^2$

$$= E(\bar{x} - E(x))^2$$

$$= E(\bar{x} - E(\bar{x}))^2$$

$$= Var(\bar{x})$$

$$= \frac{Var(x_i)}{n}$$

$$= \frac{\sigma^2}{n}$$

$$\approx \frac{s^2}{n}$$

$$= \frac{\Sigma(x_i - \bar{x})^2 / n-1}{n} = \frac{\Sigma(x_i - \bar{x})^2}{n(n-1)}$$

It depends on what you are trying to estimate (variable of interest).

If it is $\frac{D_i}{N_i}$,

Σd_i, Σn_i

d_i, n_i

Our study is finding expected value of avg time; not just the avg time.

If it were avg time by each person

1 system simulation $\Rightarrow$ $\frac{\text{Total Time}}{N}$ $\Rightarrow$ avg time by a person

But $\frac{D_i}{N_i}$ represents avg time variable.

say $\hookrightarrow Y_i$ (D_i = Total time in i^{th} simlⁿ,
N_i = -||- ppl — || —)

In the book's eg,

1 run $\rightarrow$ $\frac{\Sigma D_i}{\Sigma N_i}$ represents ? ((realized value of avg time?

If so, we just computed var, Not MSE of (Avg time)

```python
import math
import numpy as np
import sys
```

Q3

```python
def compute_mean_var_recursively(xbarj, xjp1, s2j, j):
    '''
    Given current mean, sample variance, count and the next data
point, this function finds the updated mean and sample variance
    Args:
        xbarj (float): current mean
        xjp1 (float): new data point
        s2j (float): current sample variance
        j (int): current count
    Returns:
        updated sample mean and variance (float, float)
    '''
    xbarjp1 = xbarj + (xjp1 - xbarj)/(j+1)
    s2jp1 = (1-1/j)*s2j + (j+1)*(xbarjp1 - xbarj)**2
    return xbarjp1, s2jp1

def compute_stats(X):
    '''
    Computes sample mean and variance of the data points in X
recursively (adding one element at a time)
    Args:
        X (list[float]): data points list (length = n)
    Returns:
        sample mean and variance vectors (list[float], list[float])
        (mean_vec[-1] and var_vec[-1] are the sample mean and variance
(respectively) of the entire dataset)
        (mean_vec[-2] and var_vec[-2] are the same except that they
are for X[:-1])
    '''
    n = len(X)
    mean_vec = [X[0]]
    var_vec = [0]
    for j in range(1, n):
        mean, var = compute_mean_var_recursively(mean_vec[-1], X[j],
var_vec[-1], j)
        mean_vec.append(mean)
        var_vec.append(var)
    return mean_vec, var_vec
```

Q4

(b)

```
def generate_gaussian(mu, sigma):
    '''
    Generate the realization of a gaussian random variable with mean
mu and variance sigma**2 using box muller method
    Args:
        mu (float): mean
        sigma (float): standard deviation
    Returns:
        2 iid realizations of N(mu, sigma**2) (float, float)
    '''
    u1, u2 = np.random.rand(), np.random.rand()
    v1 = 2*u1 - 1
    v2 = 2*u2 - 1
    while(v1**2 + v2**2 > 1):
        u1 = np.random.rand()
        u2 = np.random.rand()
        v1, v2 = 2*u1 - 1, 2*u2 - 1
    r2 = v1**2 + v2**2
    x = math.sqrt(-2*math.log(r2)/r2)*v1
    x = x*sigma+mu
    y = math.sqrt(-2*math.log(r2)/r2)*v2
    y = y*sigma+mu
    return x, y

def sample_std_dev(v):
    '''
    Computes the sample standard deviation of the data points in v
(assuming they are the sample)
    Args:
        v (list[float]): sample data
    Returns:
        sample std dev (float, >= 0)
    '''
    lv = len(v)
    v_mean = 0
    for i in range(lv):
        v_mean += v[i]
    v_mean /= lv
    sum = 0
    for i in range(lv):
        sum += (v[i]-v_mean)**2
    return math.sqrt(sum/(lv-1))

mu = 0
sigma = 1
a, b = generate_gaussian(mu, sigma)
```

```
data = [a, b]
n = 2
S = sample_std_dev(data)
while(S/math.sqrt(n) >= 0.1):
    if(i == 0):
        a, b = generate_gaussian(mu, sigma)
    if(i == 0):
        data.append(a)
    else:
        data.append(b)
    i = (i+1)%2
    n += 1
    S = sample_std_dev(data)

print(n)
```

```
106
```

```
# mu = 0
# sigma = 1
# data = [generate_gaussian(mu, sigma), generate_gaussian(mu, sigma)]
# n = 2
# S = sample_std_dev(data)
# while(S/math.sqrt(n) >= 0.1):
#     a = generate_gaussian(mu, sigma)
#     data.append(a)
#     n += 1
#     S = sample_std_dev(data)

# print(n)
```

(c)

```
data_mean = 0
for val in data:
    data_mean += val
data_mean /= n
print(data_mean)
```

```
0.024633943550625353
```

(d)

```
data_sample_var = 0
for val in data:
    data_sample_var += (val - data_mean)**2
data_sample_var /= (n-1)
print(data_sample_var) # This is actually S**2
```

```
1.0520302126406453
```

(e)

Not really. The results seem to be in line with expectation (mu = 0 => mean is not significantly different from 0 and sigma = 1 => sigma2 = 1 => sample variance is not significantly different from 1).

Q8

```
def get_count():
    '''
    Returns the number of U[0, 1] random variables needed to sum upto
1 (actually > 1) (int)
    '''
    ans = 0
    sum = 0
    while(sum <= 1):
        sum += np.random.rand()
        ans += 1
    return ans
```

(a)

```
num_iters = 1000
estimator_values = []
for _ in range(num_iters):
    estimator_values.append(get_count())
estimator_values = np.array(estimator_values)
e_val = np.mean(estimator_values)
print(e_val)
```

```
2.715
```

(b)

```
s2 = 0
for i in range(num_iters):
    s2 += (estimator_values[i] - e_val)**2
s2 /= (num_iters - 1)
var_estimator = s2/num_iters
print(var_estimator)
```

```
0.0007825575575575584
```

```
z_alpha_by_2 = 1.96
e_val - z_alpha_by_2*math.sqrt(var_estimator), e_val +
z_alpha_by_2*math.sqrt(var_estimator)
```

```
(np.float64(2.6601705087283025), np.float64(2.769829491271697))
```

Q9

```
def get_first_descending():
    '''
```

```
    Returns the number of U[0, 1] random variables needed until the
most recent one is smaller than its predecessor (int)
    '''
    ans = 2
    u1 = np.random.rand()
    u2 = np.random.rand()
    while(u1 <= u2):
        u1 = u2
        u2 = np.random.rand()
        ans += 1
    return ans
```

(c)

```
num_iters = 1000
estimator_values = []
for _ in range(num_iters):
    estimator_values.append(get_first_descending())
estimator_values = np.array(estimator_values)
e_val = np.mean(estimator_values)
print(e_val)
```

```
2.714
```

(d)

```
s2 = 0
for i in range(num_iters):
    s2 += (estimator_values[i] - e_val)**2
s2 /= (num_iters - 1)
var_estimator = s2/num_iters
print(var_estimator)
```

```
0.0007549589589589738
```

```
z_alpha_by_2 = 1.96
e_val - z_alpha_by_2*math.sqrt(var_estimator), e_val +
z_alpha_by_2*math.sqrt(var_estimator)
```

```
(np.float64(2.6601460276605633), np.float64(2.7678539723394366))
```

Q16

```
def simulate_single_server(T, lmda, mu):
    '''
    Simulating M/M/1/.../FIFO queue
    Args:
        T (float): total time that the system is alive (closing time
of the system assuming start time = 0)
        lmda (float): rate of arrival into the system
        g: distribution of the service time
```

```
    Returns:
        vectors containing timestamps of arrival and departure of the
people that entered into the system (list[float], list[float])
    '''

    def generate_time_of_arrival(t, lmda):
        u = np.random.rand()
        return t + 1/lmda*math.log(1/u)

    def generate_service_time(mu):
        u = np.random.rand()
        return 1/mu*math.log(1/u)

    # Start time (0), End time (T), Arrival distribution, and Service
distribution are specific to the system (they don't change with
iterations)
    # Initialization
    t = 0
    n = 0
    Na = 0 # Number of arrivals
    A = [] # Arrival times
    Nd = 0 # Number of departures
    D = [] # Departure times
    ta = generate_time_of_arrival(t, lmda)
    td = sys.maxsize # to model infinity

    # Running the system
    while(True):
        te = min(ta, td, T)
        if(te == ta): # ta = min(ta, td, T)
            t = te
            # enter only if n<=3
            if(n<=3):
                Na = Na+1
                n = n+1
                A.append(t) # A(Na) = t # A should be a vector
            # Irrespective of whether this arrival stays or not, we
need to generate time of next arrival
            ta = generate_time_of_arrival(t, lmda) # generate Tt
            if(n==1):
                delta = generate_service_time(mu)
                td = t + delta

        elif(te == td):
            t = te
            n = n-1
            Nd = Nd+1
            if(n==0):
                td = sys.maxsize
```

```
            else:
                delta = generate_service_time(mu)
                td = t + delta
            D.append(t) # D(Nd) = t # so should be D

        else: # time T reached
            while(n>0):
                t = td
                n = n-1
                Nd = Nd+1
                D.append(t)
                delta = generate_service_time(mu)
                td = t+delta
                if(n == 0):
                    td = sys.maxsize
            break

    A, D = np.array(A), np.array(D)
    return A, D
```

Method 1

```
T = 8
lmda = 3
mu = 4.2
A, D = simulate_single_server(T, lmda, mu)
avg_time = np.sum(D-A)/len(D)
print("avg time:", avg_time)

s2 = 0
lD = len(D)
time_vec = D-A
for i in range(lD):
    s2 += (time_vec[i] - avg_time)**2
s2 /= (lD-1)
mse = s2/lD
print("mse:", mse)
```

```
avg time: 0.5560401688799759
mse: 0.004018095363970494
```

Method 2

```
# Here Tvec[i] stores sum of all times of ith run i.e. sum of times
over num cycles(value of cycle) = sum over num_cycles(sum over all
people of that cycle) # 10*100 = 1000

T = 8
lmda = 3
mu = 4.2
```

```
Tvec, Nvec = [], []

num_runs = 1000
# each run has multiple cycles
for _ in range(num_runs):
    # num_iters days or cycles (as defined in the textbook)
    # 1 cycle means 1 simulation of the system
    num_cycles = 100
    sum_t_over_cycles, sum_n_over_cycles = 0, 0 # sum(di), sum(ni)
    for _ in range(num_iters):
        A, D = simulate_single_server(T, lmda, mu) # these give d1, n1
i.e. numbers for a single cycle
        sum_t_over_cycles += np.sum(D-A) # di
        sum_n_over_cycles += len(D) # ni
    Tvec.append(sum_t_over_cycles)
    Nvec.append(sum_n_over_cycles)

sum_t, sum_n = 0, 0
for i in range(num_runs):
    sum_t += Tvec[i]
    sum_n += Nvec[i]
avg_t_by_avg_n = sum_t/sum_n
print("avg time:", avg_t_by_avg_n)

sse = 0
for i in range(num_runs):
    sse += (Tvec[i]/Nvec[i] - avg_t_by_avg_n)**2 # (X_bar -
E(X_bar))**2
mse = sse/num_runs
print("mse:", mse)
```

```
avg time: 0.48019758014570346
mse: 0.00029807888573462195
```

Unsuccessful tries

```
### No num_runs, only num_iters (Here, Tvec[i] = total time (sum of
all times) of ith cycle) # 10

# # Initialization
# T = 8
# lmda = 3
# mu = 4.2
# Tvec, Nvec = [], []

# # num_iters days or cycles (as defined in the textbook)
# num_iters = 100
# for _ in range(num_iters):
#     A, D = simulate_single_server(T, lmda, mu)
#     Tvec.append(np.sum(D-A))
```

```
#     Nvec.append(len(D))

# sum_t, sum_n = 0, 0
# for i in range(num_iters):
#     sum_t += Tvec[i]
#     sum_n += Nvec[i]
# avg_t_by_avg_n = sum_t/sum_n
# print("avg time:", avg_t_by_avg_n)

# sse = 0
# for i in range(num_iters):
#     sse += (Tvec[i]/Nvec[i] - avg_t_by_avg_n)**2 # (X_bar -
E(X_bar))**2
# mse = sse/num_iters
# print("mse:", mse)

### avg_t_by_n

# # Initialization
# T = 8
# lmda = 3
# mu = 4.2
# Tvec, Nvec = [], []
# avg_t_by_n = 0

# # num_iters days or cycles (as defined in the textbook)
# num_iters = 100
# for _ in range(num_iters):
#     A, D = simulate_single_server(T, lmda, mu)
#     v1 = np.sum(D-A)
#     v2 = len(D)
#     Tvec.append(v1)
#     Nvec.append(v2)
#     avg_t_by_n += v1/v2
# avg_t_by_n /= num_iters
# print("avg time:", avg_t_by_n)

# sse = 0
# for i in range(num_iters):
#     sse += (Tvec[i]/Nvec[i] - avg_t_by_n)**2 # (X_bar - E(X_bar))**2
# mse = sse/num_iters
# print("mse:", mse)
```